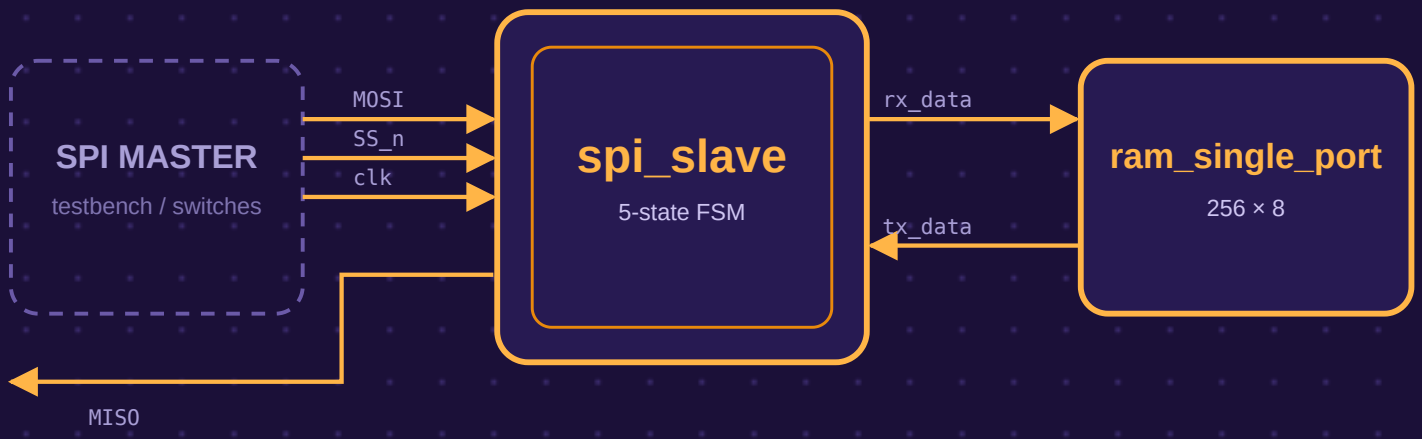


DIGITAL DESIGN PROJECT

SPI Slave with Single-Port RAM

Verilog Design & Implementation

RTL, testbench, simulation, linting, synthesis and implementation results for three FSM encodings.

TEAM
NO Signal

MEMBERS
Ahmed Hazem El-Sehly
Ahmed Mohamed Khalafallah

LANGUAGE
Verilog HDL

TOOLS
QuestaSim · QuestaLint · Vivado

DEVICE
xc7a35ticpg236-1L

Contents

01 Design Overview **3**

02 Simulation Waveforms **5**

03 RTL Code **6**

ram_single_port 6

spi_slave 7

spi_ram_top (top module) 10

04 Testbench **11**

Test plan and run.do 11

Testbench code, part 1 12

Testbench code, part 2 13

05 Linting **14**

06 Elaboration & Synthesis **15**

6.1 One-hot 16

6.2 Gray 18

6.3 Sequential 20

07 Implementation **22**

7.1 One-hot 23

7.2 Gray 25

7.3 Sequential 27

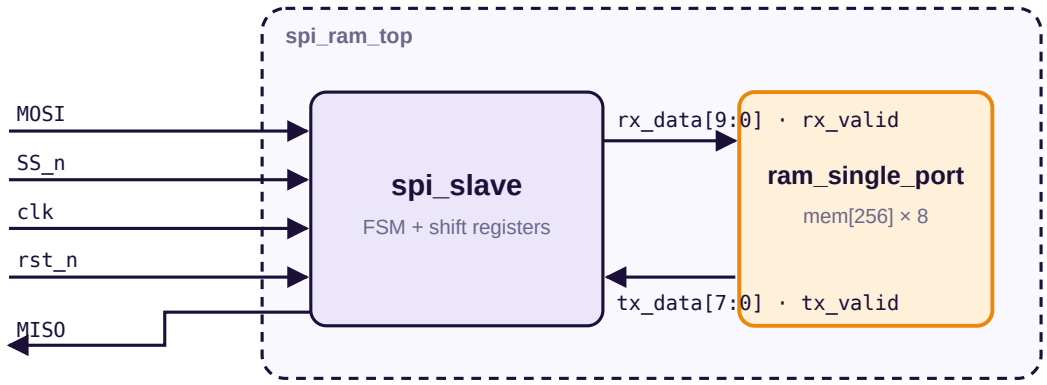
08 FSM Encoding Selection **29**

01 Design Overview

SPI slave controlling a single-port RAM

This report documents a Verilog implementation of an **SPI slave** connected to a **single-port synchronous RAM** (256 × 8). The slave receives 10-bit frames on **MOSI** (MSB first) while **SS_n** is low. The two upper bits select the operation and the lower eight bits carry an address or a data byte. For a read, the RAM returns the byte and the slave shifts it out on **MISO**. The FSM of the slave was synthesised with three encodings (one-hot, gray, sequential) and each one was taken through synthesis and implementation in Vivado, after simulation in QuestaSim and a QuestaLint check.

Top-level block diagram



Module hierarchy

Module	Role
`spi_ram_top`	Top level. Connects spi_slave and ram_single_port; ports clk, rst_n, MOSI, SS_n, MISO.
`spi_slave`	SPI slave FSM (5 states), 10-bit MOSI shift register, 8-bit MISO shift register, bit counter and read-phase control.
`ram_single_port`	256 × 8 memory with write-address and read-address registers; decodes the 2-bit command in din[9:8].

Command frame (10 bits, MSB first)

din[9:8]	Command	RAM action
`00`	Write address	addr_w ← din[7:0]
`01`	Write data	mem[addr_w] ← din[7:0]
`10`	Read address	addr_r ← din[7:0]
`11`	Read data	dout ← mem[addr_r], tx_valid = 1; the slave then shifts dout out on MISO

FSM states of spi_slave

State	Behaviour
S_IDLE	Waits for SS_n = 0. Clears the bit counter, the read phase and rx_done.
S_CHK_CMD	Samples the first MOSI bit: 0 → S_WRITE; 1 → S_READ_DATA if an address is already pending (addr_ready), otherwise S_READ_ADD. Returns to S_IDLE if SS_n is high.
S_WRITE	Shifts in the remaining bits of the frame, then pulses rx_valid with rx_data (write-address or write-data command).
S_READ_ADD	Same as S_WRITE for the read-address command; also sets addr_ready.
S_READ_DATA	Receives the read-data frame, waits for tx_valid from the RAM, then shifts the 8-bit result out on MISO, MSB first.

All states return to S_IDLE when SS_n goes high.

State encodings reported by Vivado

State	Original (RTL)	One-hot	Gray	Sequential
S_IDLE	000	00001	000	000
S_CHK_CMD	001	00010	001	001
S_WRITE	010	00100	011	010
S_READ_DATA	100	01000	010	011
S_READ_ADD	011	10000	111	100

Results at a glance

Encoding	State bits	Synth WNS	Synth WHS	Impl WNS	Impl WHS	Total LUT	Total FF
One-hot	5	6.418 ns	0.139 ns	**2.812 ns**	0.012 ns	1289	1973
Gray	3	6.471 ns	0.154 ns	1.789 ns	0.029 ns	1293	1971
Sequential	3	6.471 ns	0.139 ns	1.725 ns	0.068 ns	1292	1971

Clock constraint: 10.00 ns period on port `clk`. All user-specified timing constraints are met for all three encodings. Total LUT/FF include the Vivado debug cores (`dbg_hub`, `u_ila_0`) visible in the utilization hierarchy; the `spi_slave` instance alone uses 49 LUT / 40 FF (one-hot) and 53 LUT / 38 FF (gray and sequential). Device: xc7a35ticpg236-1L.

02 Simulation Waveforms

QuestaSim wave window, 0 to 670 ns

Waveform with input values assigned and outputs visible

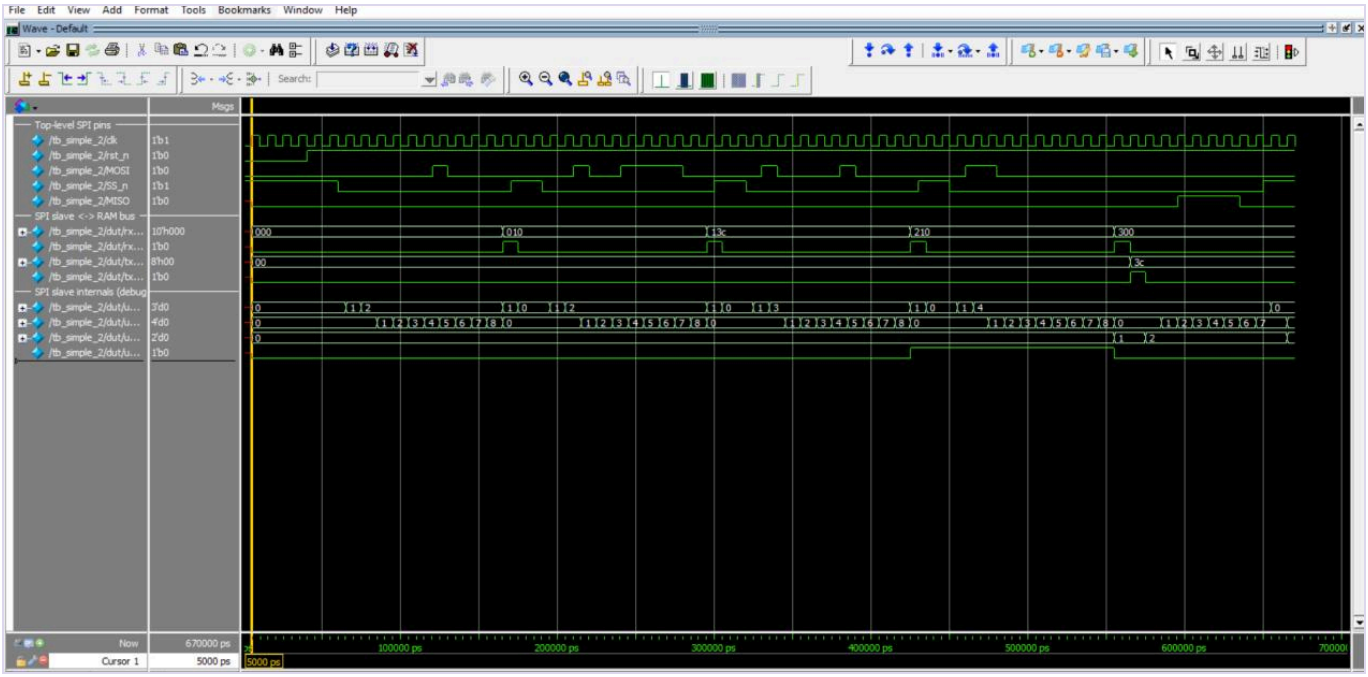


Figure 1. *QuestaSim waveform of the testbench run (clk, rst_n, MOSI, SS_n, MISO; rx_data / rx_valid / tx_data / tx_valid; state, bit_cnt, rd_phase, addr_ready).*

Four frames are sent with `SS_n` low during each one. The slave-to-RAM bus shows `rx_data` = **010** (write address 0x10), **13c** (write data 0x3C), **210** (read address 0x10) and **300** (read-data command). After the last frame the RAM returns `tx_data` = **3c** with `tx_valid` high and the byte is shifted out on `MISO`.

Transcript

```
SIM 20> run -all
# PASS: read back 0x3c from address 0x10 (expected 0x3C)
# ** Note: $stop    : tb_simple_2.v(117)
#    Time: 670 ns  Iteration: 1  Instance: /tb_simple_2
# Break in Module tb_simple_2 at tb_simple_2.v line 117
```

Figure 2. *Transcript: the read-back value matches the expected value (simulation stops at 670 ns).*

Revision note: rx_done

The register `rx_done` in `spi_slave.v` was added recently, after the waveform, lint, elaboration, synthesis and implementation snippets in this report were captured. It is therefore **not** among the signals probed in the waveform above (see the signal list in `run.do`) and does not appear in the schematics or reports of the following sections; those snippets show the revision without it. Its job is to stop the shift register from capturing more bits once the 10-bit frame has been received in S_WRITE / S_READ_ADD; it is cleared in S_IDLE.

03 RTL Code

Verilog source for every module, bottom-up

ram_single_port

Single-port RAM. The two upper bits of din select the operation: write address, write data, read address or read data.

ram_single_port.v · 33 lines

```
1  module ram_single_port(din,rx_valid,clk,rst_n,dout,tx_valid);
2      parameter ADDER_SIZE = 8;
3      parameter MEM_DEPTH = 256;
4      localparam IN_WIDTH = ADDER_SIZE +2;
5      input [IN_WIDTH-1:0] din;
6      input rx_valid,clk,rst;
7      output reg [ADDER_SIZE-1:0] dout;
8      output reg tx_valid;
9      reg [ADDER_SIZE-1:0] mem [MEM_DEPTH-1:0];
10     reg [ADDER_SIZE-1:0] addr_r;
11     reg [ADDER_SIZE-1:0] addr_w;
12     always @(posedge clk) begin
13         if(~rst_n) begin
14             addr_w <= 'b0;
15             addr_r <= 'b0;
16             dout   <= 'b0;
17             tx_valid <= 1'b0;
18         end else begin
19             tx_valid <= 1'b0;
20             if (rx_valid)
21             case (din[IN_WIDTH-1:IN_WIDTH-2])
22                 2'b00:addr_w <= din[IN_WIDTH-3:0];
23                 2'b01:mem[addr_w] <= din[IN_WIDTH-3:0];
24                 2'b10:addr_r <= din[IN_WIDTH-3:0];
25                 2'b11: begin
26                     dout <= mem[addr_r];
27                     tx_valid <= 1'b1;
28                 end
29                 default: ;
30             endcase
31         end
32     end
33 endmodule //ram
```

Revision note: rx_done

The code in this section is the current version of the files, including `rx_done` in `spi_slave.v` (lines 34, 76, 88, 100 and 105). The simulation, lint, synthesis and implementation snippets in the other sections were captured on the revision before `rx_done` was added.

spi_slave (part 1: declarations and FSM)

SPI slave. 3-block FSM: state memory, next-state logic and output/datapath logic.

spi_slave.v · lines 1–52

```
1  module spi_slave #(
2      parameter ADDR_SIZE = 8
3  )(
4      input  wire        clk,
5      input  wire        rst_n,
6      input  wire        MOSI,
7      input  wire        SS_n,
8      output wire         MISO,
9
10     //interface to the RAM
11     output reg  [ADDR_SIZE+1:0]   rx_data,
12     output reg           rx_valid,
13     input  wire [ADDR_SIZE-1:0]   tx_data,
14     input  wire          tx_valid
15 );
16     localparam CNT_W = $clog2(ADDR_SIZE);
17     // State encoding
18     // one_hot / gray / sequential
19     localparam [2:0]
20         S_IDLE      = 3'd0,
21         S_CHK_CMD   = 3'd1,
22         S_WRITE     = 3'd2,
23         S_READ_ADD  = 3'd3,
24         S_READ_DATA = 3'd4;
25
26     (* fsm_encoding = "one_hot" *)
27     reg [2:0] state, next_state;
28
29     reg [ADDR_SIZE+1:0] shift_in;    // serial -> parallel (MOSI) shift register
30     reg [ADDR_SIZE-1:0] shift_out;   // parallel -> serial (MISO) shift register
31     reg [CNT_W:0] bit_cnt;
32     reg [1:0] rd_phase;
33     reg       addr_ready;
34     reg       rx_done;
35
36     // 1) STATE MEMORY (sequential)
37     always @(posedge clk) begin
38         if (!rst_n)
39             state <= S_IDLE;
40         else
41             state <= next_state;
42     end
43
44     // 2) NEXT-STATE LOGIC (combinational)
45     always @(*) begin
46         next_state = state;
47         case (state)
48             S_IDLE: begin
49                 if (!SS_n)
50                     next_state = S_CHK_CMD;
51             end
52
```

spi_slave (part 2: shift logic)

spi_slave.v · lines 53–104

```
53              S_CHK_CMD: begin
54                  if (SS_n)
55                      next_state = S_IDLE;
56                  else if (MOSI == 1'b0)
57                      next_state = S_WRITE;
58                  else
59                      next_state = addr_ready ? S_READ_DATA : S_READ_ADD;
60              end
61              S_WRITE, S_READ_ADD, S_READ_DATA: begin
62                  if (SS_n)
63                      next_state = S_IDLE;
64              end
65              default: next_state = S_IDLE;
66          endcase
67      end
68
69      // 3) OUTPUT
70      always @(posedge clk) begin
71          if (!rst_n) begin
72              shift_in   <= {(ADDR_SIZE+2){1'b0}};
73              shift_out  <= {ADDR_SIZE{1'b0}};
74              bit_cnt    <= 'd0;
75              rd_phase   <= 2'd0;
76              rx_done    <= 1'b0;
77              addr_ready <= 1'b0;
78              rx_data    <= {(ADDR_SIZE+2){1'b0}};
79              rx_valid   <= 1'b0;
80          end else begin
81              rx_valid <= 1'b0;
82
83              case (state)
84
85                  S_IDLE: begin
86                      bit_cnt  <= 'd0;
87                      rd_phase <= 2'd0;
88                      rx_done  <= 1'b0;
89                  end
90
91                  S_CHK_CMD: begin
92                      if (!SS_n) begin
93                          shift_in <= {{(ADDR_SIZE+1){1'b0}}, MOSI};
94                          bit_cnt  <= 'd0;
95                          rd_phase <= 2'd0;
96                      end
97                  end
98
99                  S_WRITE, S_READ_ADD: begin
100                     if (!SS_n && !rx_done) begin
101                         shift_in <= {shift_in[ADDR_SIZE:0], MOSI};
102                         if (bit_cnt == ADDR_SIZE) begin
103                             rx_data  <= {shift_in[ADDR_SIZE:0], MOSI};
104                             rx_valid <= 1'b1;
```

spi_slave (part 3: read-data phases and MISO)

spi_slave.v · lines 105–156

```
105                             rx_done  <= 1'b1;
106                             bit_cnt  <= 'd0;
107                             if (state == S_READ_ADD)
108                                 addr_ready <= 1'b1;   // address now pending in RAM
109                         end else begin
110                             bit_cnt <= bit_cnt + 1'b1;
111                         end
112                     end
113                 end
114 
115                 // -----------------------------------------------------
116                 S_READ_DATA: begin
117                     case (rd_phase)
118                         2'd0: begin
119                             if (!SS_n) begin
120                                 shift_in <= {shift_in[ADDR_SIZE:0], MOSI};
121                                 if (bit_cnt == ADDR_SIZE) begin
122                                     rx_data    <= {shift_in[ADDR_SIZE:0], MOSI};
123                                     rx_valid   <= 1'b1;
124                                     addr_ready <= 1'b0;  // address now consumed
125                                     bit_cnt    <= 'd0;
126                                     rd_phase   <= 2'd1;
127                                 end else begin
128                                     bit_cnt <= bit_cnt + 1'b1;
129                                 end
130                             end
131                         end
132                         2'd1: begin // waiting for the RAM to return the data
133                             if (tx_valid) begin
134                                 shift_out <= tx_data;
135                                 bit_cnt   <= 'd0;
136                                 rd_phase  <= 2'd2;
137                             end
138                         end
139                         2'd2: begin // shifting the byte out on MISO, MSB first
140                             if (!SS_n && bit_cnt < ADDR_SIZE) begin
141                                 shift_out <= {shift_out[ADDR_SIZE-2:0], 1'b0};
142                                 bit_cnt   <= bit_cnt + 1'b1;
143                             end
144                         end
145                         default: ;
146                     endcase
147                 end
148 
149                 default: ;
150             endcase
151         end
152     end
153 
154     assign MISO = (state == S_READ_DATA && rd_phase == 2'd2) ? shift_out[ADDR_SIZE-1] : 1'b0;
155 
156 endmodule
```

spi_ram_top (top module)

Top level. Instantiates spi_slave and ram_single_port and wires the rx/tx bus between them.

spi_ram_top.v · 44 lines

```
module spi_ram_top #(
    parameter MEM_DEPTH = 256,
    parameter ADDR_SIZE = 8
)(
    input  wire clk,
    input  wire rst_n,
    input  wire MOSI,
    input  wire SS_n,
    output wire MISO
);

    // internal SPI-slave <-> RAM bus
    wire [ADDR_SIZE+1:0] rx_data;
    wire       rx_valid;
    wire [ADDR_SIZE-1:0] tx_data;
    wire       tx_valid;

    spi_slave #(
        .ADDR_SIZE (ADDR_SIZE)
    )u_spi_slave (
        .clk      (clk),
        .rst_n    (rst_n),
        .MOSI     (MOSI),
        .SS_n     (SS_n),
        .MISO     (MISO),
        .rx_data  (rx_data),
        .rx_valid (rx_valid),
        .tx_data  (tx_data),
        .tx_valid (tx_valid)
    );

    ram_single_port #(
        .MEM_DEPTH (MEM_DEPTH),
        .ADDR_SIZE (ADDR_SIZE)
    ) u_ram (
        .clk      (clk),
        .rst_n    (rst_n),
        .din      (rx_data),
        .rx_valid (rx_valid),
        .dout     (tx_data),
        .tx_valid (tx_valid)
    );

endmodule
```

04 Testbench

Test plan and testbench code

Test plan

The testbench drives the SPI pins of `spi_ram_top` with a 10 ns clock (`always #5 clk = ~clk`). It resets the design, writes a byte into the RAM and reads it back over MISO. Bits are changed on the falling clock edge, one bit per clock, and the captured byte is compared with the expected value.

Step	Frame (din)	Action	Expected
0 Reset	—	rst_n = 0 for 4 negedges, then released; 2 more negedges	Slave in S_IDLE, outputs idle
1	00 · 0x10	Write address = 0x10	rx_data = 010
2	01 · 0x3C	Write data 0x3C into the stored address	rx_data = 13c
3	10 · 0x10	Read address = 0x10	rx_data = 210
4	11 · 0x00	Read data; one extra clock, then 8 bits are captured from MISO	received_byte = 0x3C

Each frame is sent with SS_n low and followed by SS_n high and two idle negedges. The testbench prints `PASS` or `FAIL` and calls `$stop`.

Simulation script (run.do)

run.do · 26 lines

```
vlib work
vlog -sv ram_single_port.v spi_slave.v spi_ram_top.v tb_simple_2.v

vsim -voptargs=+acc work.tb_simple_2

add wave -divider "Top-level SPI pins"
add wave -radix binary   tb_simple_2/clk
add wave -radix binary   tb_simple_2/rst_n
add wave -radix binary   tb_simple_2/MOSI
add wave -radix binary   tb_simple_2/SS_n
add wave -radix binary   tb_simple_2/MISO

add wave -divider "SPI slave <-> RAM bus"
add wave -radix hex      tb_simple_2/dut/rx_data
add wave -radix binary   tb_simple_2/dut/rx_valid
add wave -radix hex      tb_simple_2/dut/tx_data
add wave -radix binary   tb_simple_2/dut/tx_valid

add wave -divider "SPI slave internals (debug)"
add wave -radix unsigned tb_simple_2/dut/u_spi_slave/state
add wave -radix unsigned tb_simple_2/dut/u_spi_slave/bit_cnt
add wave -radix unsigned tb_simple_2/dut/u_spi_slave/rd_phase
add wave -radix binary   tb_simple_2/dut/u_spi_slave/addr_ready

run -all
wave zoom full
```

Testbench code, part 1: setup, reset and write frames (steps 1–2)

tb_simple_2.v · lines 1–66

```
1  `timescale 1ns/1ps
2
3  module tb_simple_2;
4
5      reg  clk   = 0;
6      reg  rst_n = 0;
7      reg  MOSI  = 0;
8      reg  SS_n  = 1;
9      wire MISO;
10
11     // temporary variable
12     reg [9:0] frame;
13     integer   i;
14
15     reg [7:0] received_byte;
16
17     // Instantiate
18     spi_ram_top dut (
19         .clk   (clk),
20         .rst_n (rst_n),
21         .MOSI  (MOSI),
22         .SS_n  (SS_n),
23         .MISO  (MISO)
24     );
25
26     always #5 clk = ~clk;
27
28     initial begin
29
30         // Step 0: reset
31         rst_n = 0;
32         SS_n  = 1;
33         MOSI  = 0;
34         repeat (4) @(negedge clk);
35         rst_n = 1;
36         repeat (2) @(negedge clk);
37
38
39         // Step 1: send "store write-address = 0x10"
40         // frame = 00_00010000
41         frame = {2'b00, 8'h10};
42         SS_n  = 0;
43         @(negedge clk);
44
45         for (i = 9; i >= 0; i = i - 1) begin
46             MOSI = frame[i];      // send one bit per clock
47             @(negedge clk);
48         end
49
50         SS_n = 1;
51         repeat (2) @(negedge clk);
52
53         // Step 2: send "write 0x3C into the stored address"
54         // frame = 01_00111100
55         frame = {2'b01, 8'h3C};
56         SS_n  = 0;
57         @(negedge clk);
58
59         for (i = 9; i >= 0; i = i - 1) begin
60             MOSI = frame[i];
61             @(negedge clk);
62         end
63
64         SS_n = 1;
65         repeat (2) @(negedge clk);
66
```

Testbench code, part 2: read-address, read-data frames and check (steps 3–4)

tb_simple_2.v · lines 67–116

```
67          // Step 3: send "store read-address = 0x10"
68          // frame = 10_00010000
69          frame = {2'b10, 8'h10};
70          SS_n  = 0;
71          @(negedge clk);
72
73          for (i = 9; i >= 0; i = i - 1) begin
74              MOSI = frame[i];
75              @(negedge clk);
76          end
77
78          SS_n = 1;
79          repeat (2) @(negedge clk);
80
81
82          // Step 4: send "give me the data" and capture the reply on MISO
83          // frame = 11_00000000 (the last 8 bits are dummy)
84          frame = {2'b11, 8'h00};
85          SS_n  = 0;
86          @(negedge clk);
87
88          for (i = 9; i >= 0; i = i - 1) begin
89              MOSI = frame[i];
90              @(negedge clk);
91          end
92
93          // after sending the frame, the RAM needs one extra clock to
94          // register its reply (tx_valid/tx_data)
95          @(negedge clk);
96
97          // MISO now starts shifting out 8 bits, capture them one by one
98          received_byte = 8'h00;
99          for (i = 0; i < 8; i = i + 1) begin
100             @(negedge clk);
101             received_byte = {received_byte[6:0], MISO};
102         end
103
104         SS_n = 1;
105         repeat (2) @(negedge clk);
106
107         // Print the result and compare against the expected value
108         if (received_byte === 8'h3C)
109             $display("PASS: read back 0x%0h from address 0x10 (expected 0x3C)", received_byte);
110         else
111             $display("FAIL: read back 0x%0h, expected 0x3C", received_byte);
112
113         $stop;
114     end
115
116 endmodule
```

05 Linting

QuestaLint (Questa Static Verification, v2021.1)

Item	Result
Design	spi_ram_top
Errors	0
Warnings	0
Info	3 (parameter_name_duplicate × 2, always_signal_assign_large × 1)
Design Quality Score	100%

The three Info items are naming / style checks and can be waived.

Lint Checks window: Info-level items only

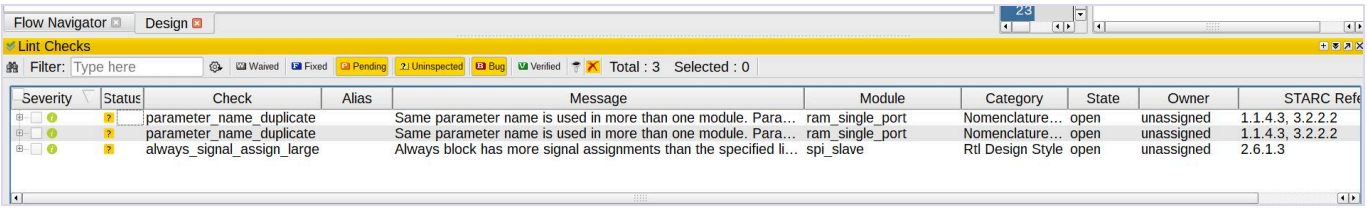


Figure 3. *Lint Checks: total 3, all Info (two `parameter_name_duplicate`, one `always_signal_assign_large`). Not needed — can be waived.*

Lint Checks window: no violations

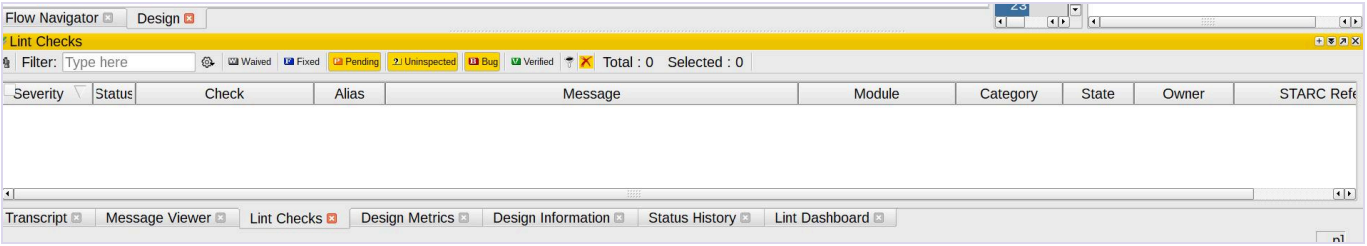


Figure 4. *Lint Checks with no remaining items: no linting errors.*

06 Elaboration & Synthesis

Vivado RTL analysis and synthesised design, per FSM encoding

The same RTL was elaborated and synthesised three times, once per FSM encoding. For each encoding this section shows the elaborated schematic, the synthesised schematic, the Vivado synthesis log line with the encoding that was used, the design timing summary and the critical path highlighted in the schematic.

Post-synthesis timing summary

Encoding	WNS (setup)	WHS (hold)	WPWS	Failing endpoints
One-hot	6.418 ns	0.139 ns	4.500 ns	0
Gray	6.471 ns	0.154 ns	4.500 ns	0
Sequential	6.471 ns	0.139 ns	4.500 ns	0

All user-specified timing constraints are met. The synthesised design has 8 cells, 5 I/O ports and 34 nets for each encoding.

Revision note: rx_done

All elaboration and synthesis snippets were captured before `rx_done` was added to `spi_slave.v`, so the schematics and reports show the revision without it.

6.1 One-hot: elaborated and synthesised schematics

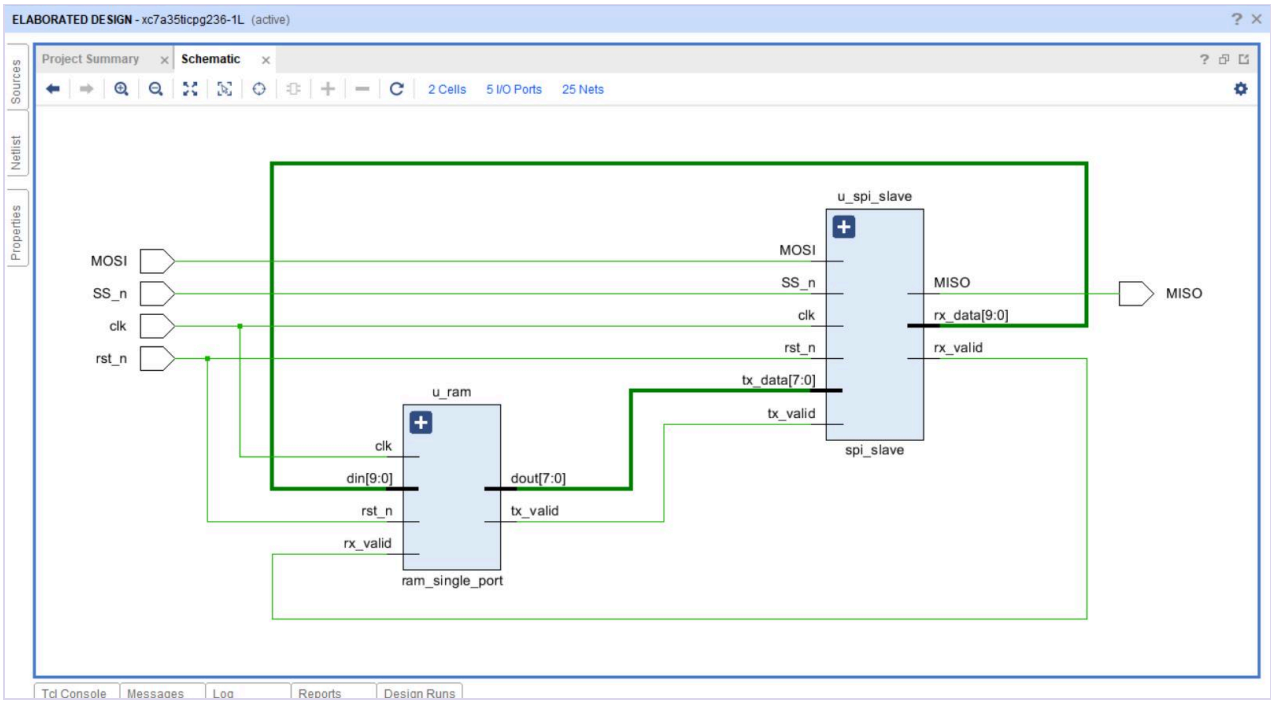


Figure 5. *One-hot: schematic after elaboration (2 cells, 5 I/O ports, 25 nets).*

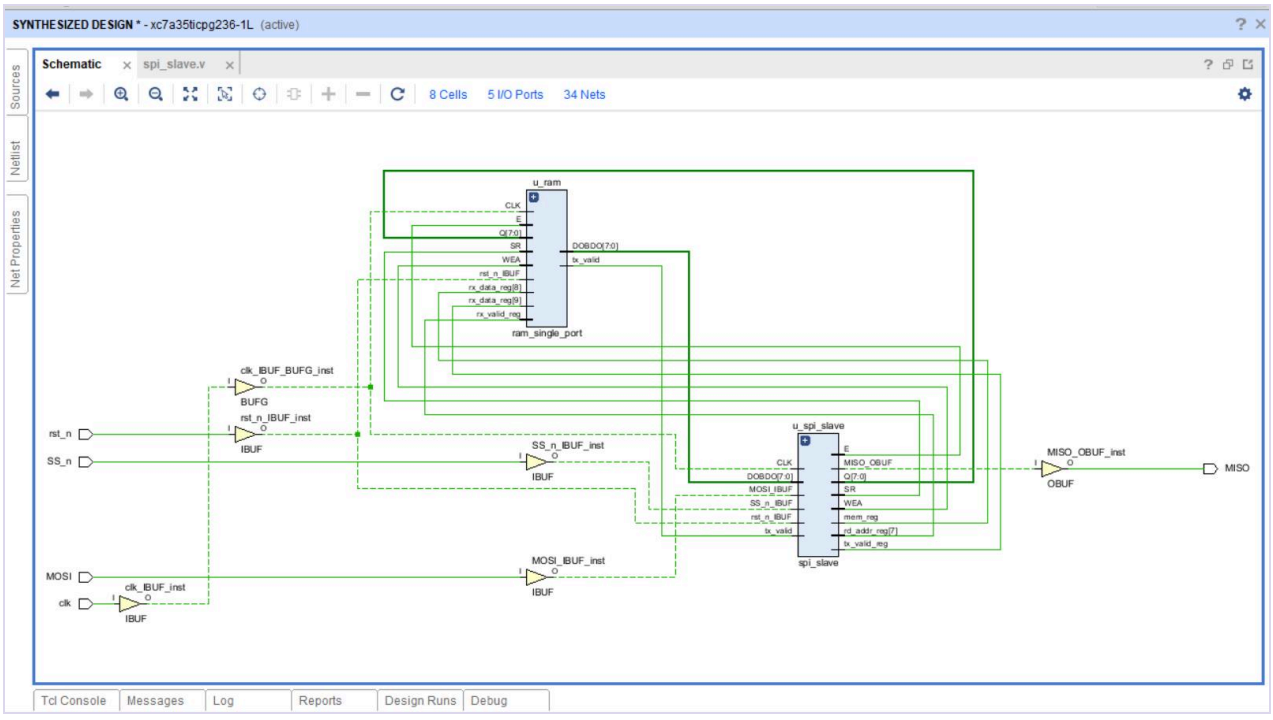


Figure 6. *One-hot: schematic after synthesis (8 cells, 5 I/O ports, 34 nets).*

6.1 One-hot: synthesis report, timing and critical path

```
99  ---------------------------------------------------------------------------------------------------
100 |                State |                New Encoding |                Previous Encoding
101 ---------------------------------------------------------------------------------------------------
102 |              S_IDLE |                       00001 |                             000
103 |           S_CHK_CMD |                       00010 |                             001
104 |             S_WRITE |                       00100 |                             010
105 |         S_READ_DATA |                       01000 |                             100
106 |          S_READ_ADD |                       10000 |                             011
107 ---------------------------------------------------------------------------------------------------
108 INFO: [Synth 8-3354] encoded FSM with state register 'state_reg' using encoding 'one-hot' in module 'spi_slave'
109 ---------------------------------------------------------------------------------------------------
```

Figure 7. *One-hot: synthesis log showing the FSM encoding used (`'one-hot'` for `state_reg` in `spi_slave`).*

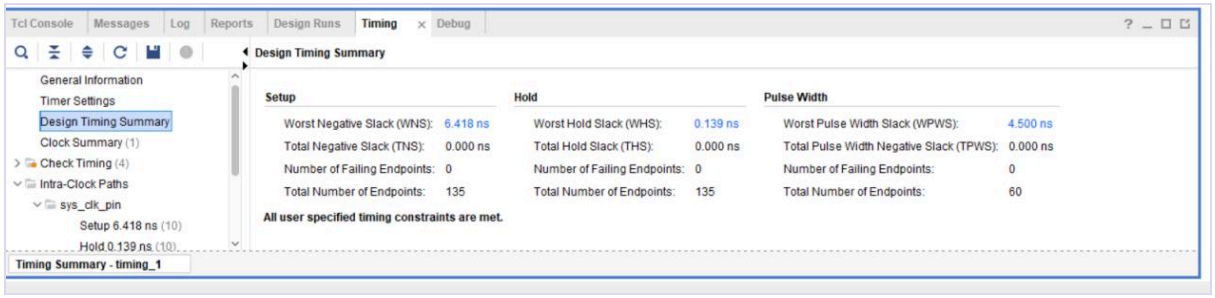


Figure 8. *One-hot: post-synthesis design timing summary.*

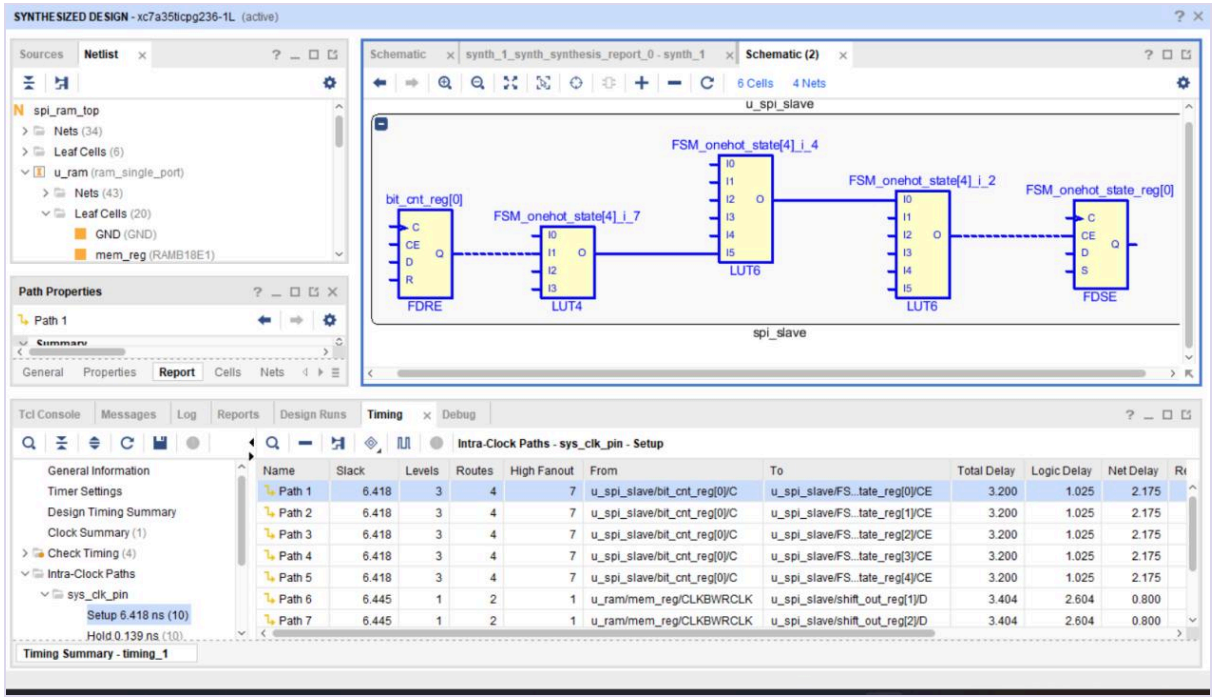


Figure 9. *One-hot: critical path highlighted in the schematic, with the intra-clock path report.*

6.2 Gray: elaborated and synthesised schematics

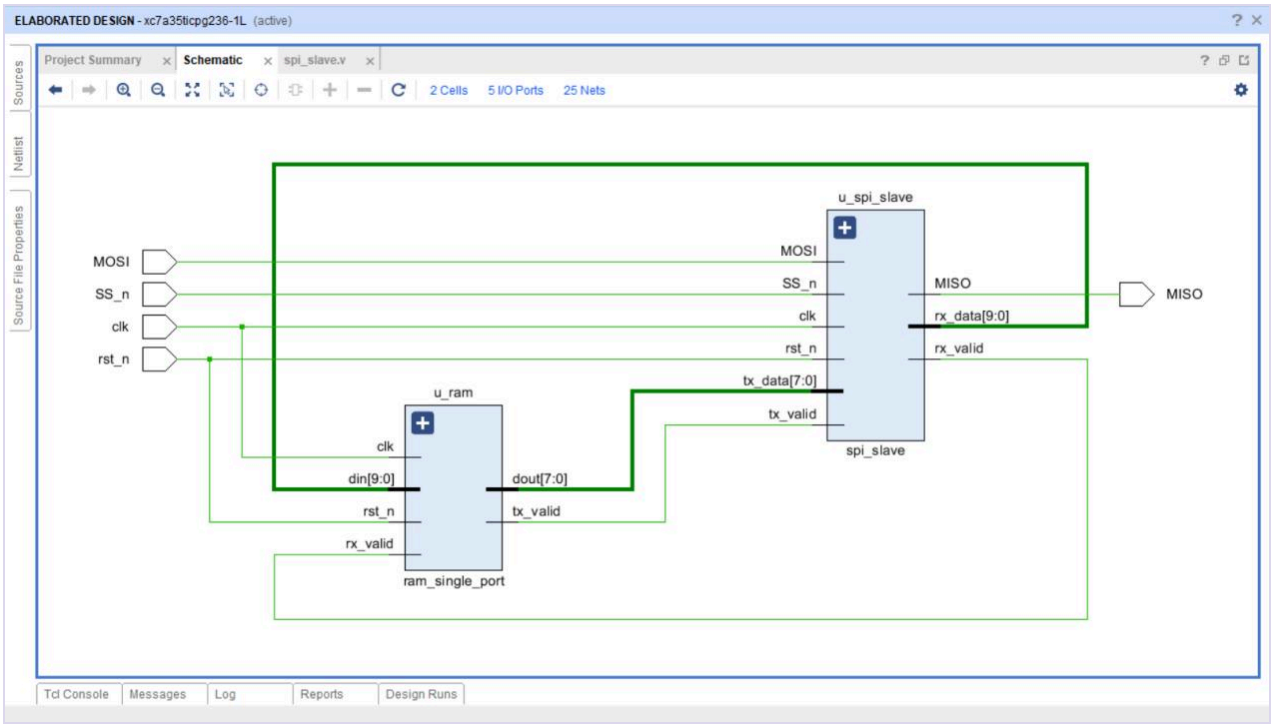


Figure 10. *Gray: schematic after elaboration (2 cells, 5 I/O ports, 25 nets).*

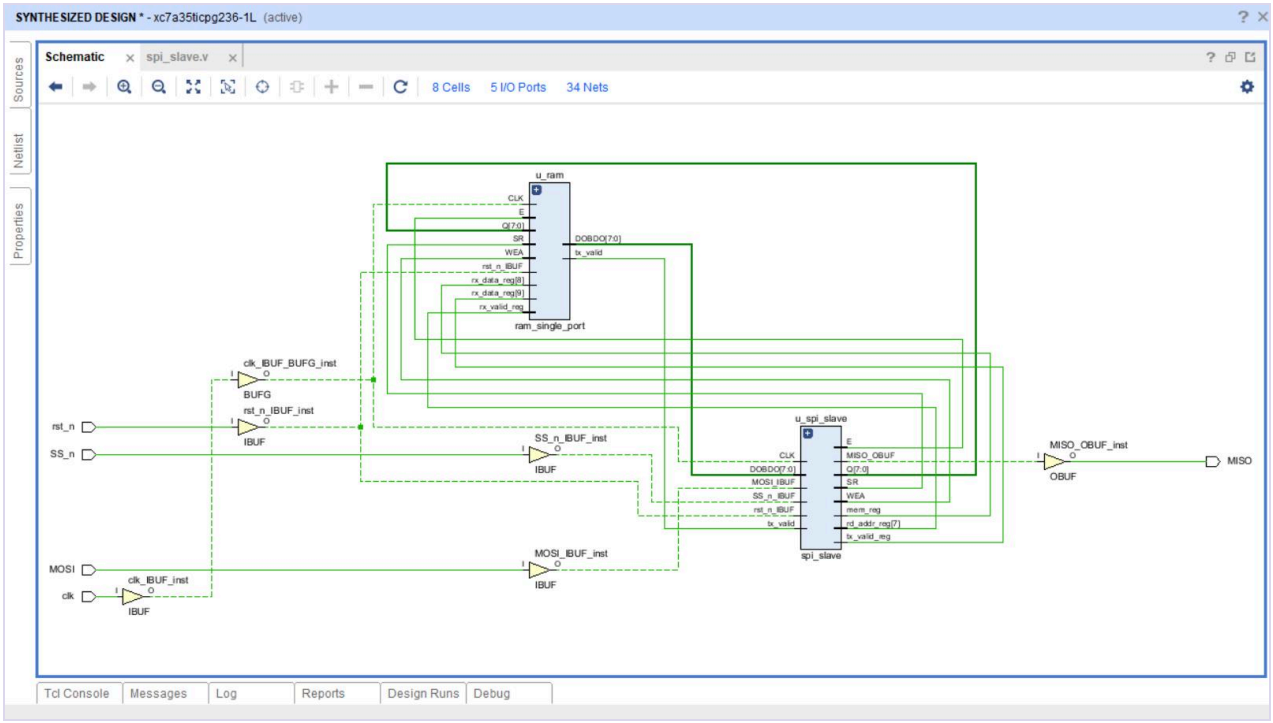


Figure 11. *Gray: schematic after synthesis (8 cells, 5 I/O ports, 34 nets).*

6.2 Gray: synthesis report, timing and critical path

```
99  ---------------------------------------------------------------------------------------------------
100 |                State |                New Encoding |                Previous Encoding
101 ---------------------------------------------------------------------------------------------------
102 |               S_IDLE |                         000 |                             000
103 |            S_CHK_CMD |                         001 |                             001
104 |              S_WRITE |                         011 |                             010
105 |          S_READ_DATA |                         010 |                             100
106 |           S_READ_ADD |                         111 |                             011
107 ---------------------------------------------------------------------------------------------------
108 INFO: [Synth 8-3354] encoded FSM with state register 'state_reg' using encoding 'gray' in module 'spi_slave'
109 ---------------------------------------------------------------------------------------------------
```

Figure 12. *Gray: synthesis log showing the FSM encoding used (`'gray'` for `state_reg` in `spi_slave`).*

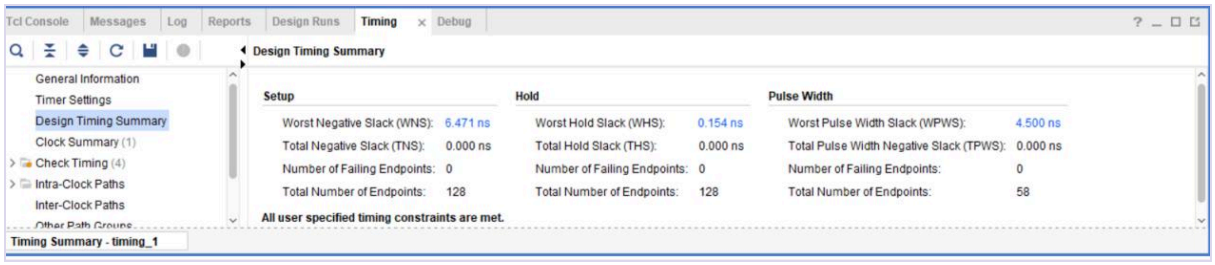


Figure 13. *Gray: post-synthesis design timing summary.*

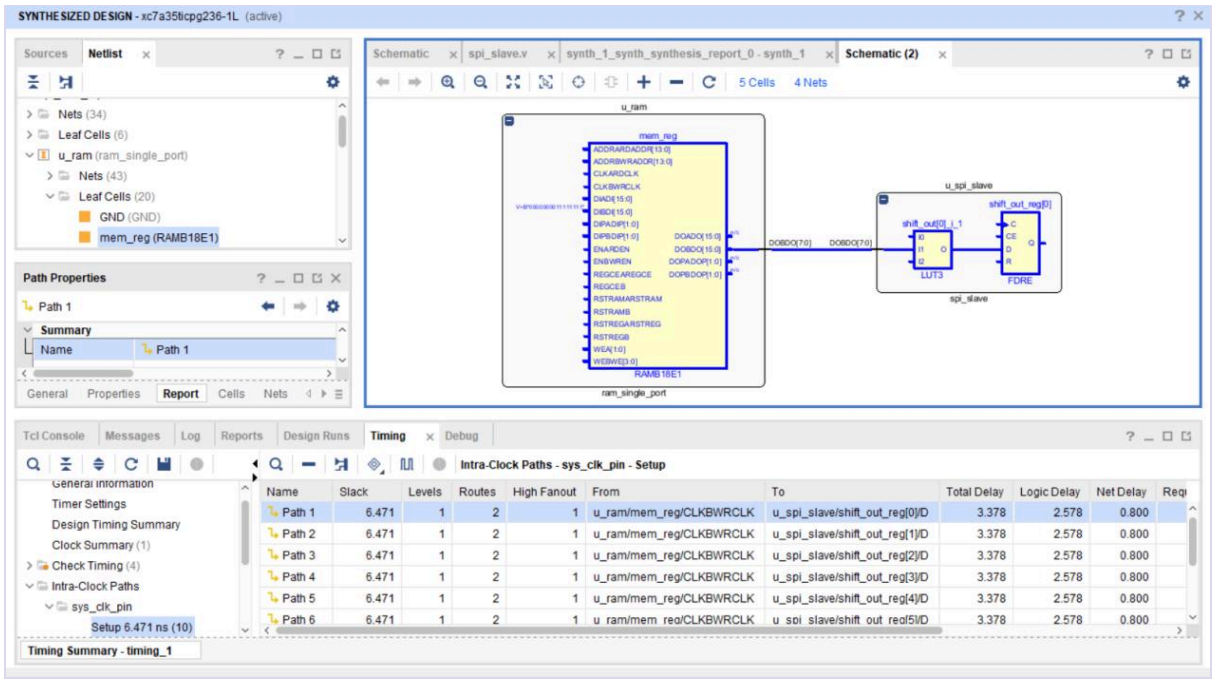


Figure 14. *Gray: critical path highlighted in the schematic, with the intra-clock path report.*

6.3 Sequential: elaborated and synthesised schematics

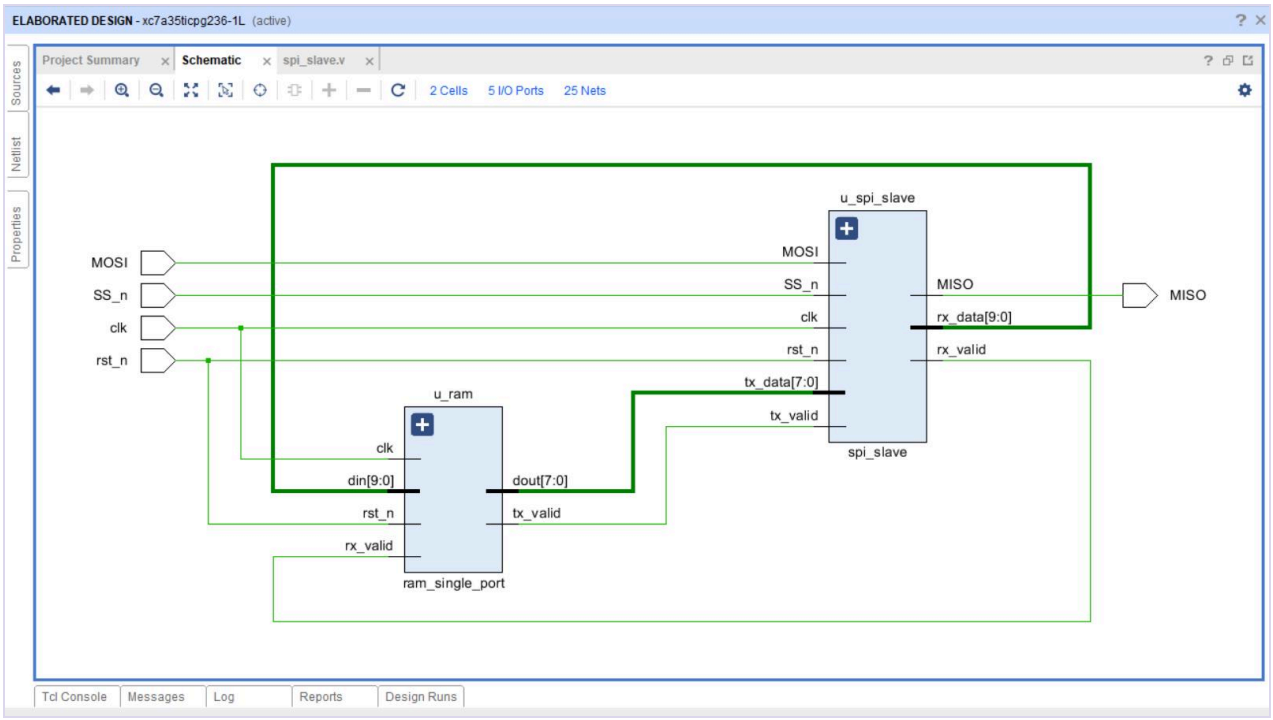


Figure 15. *Sequential: schematic after elaboration (2 cells, 5 I/O ports, 25 nets).*

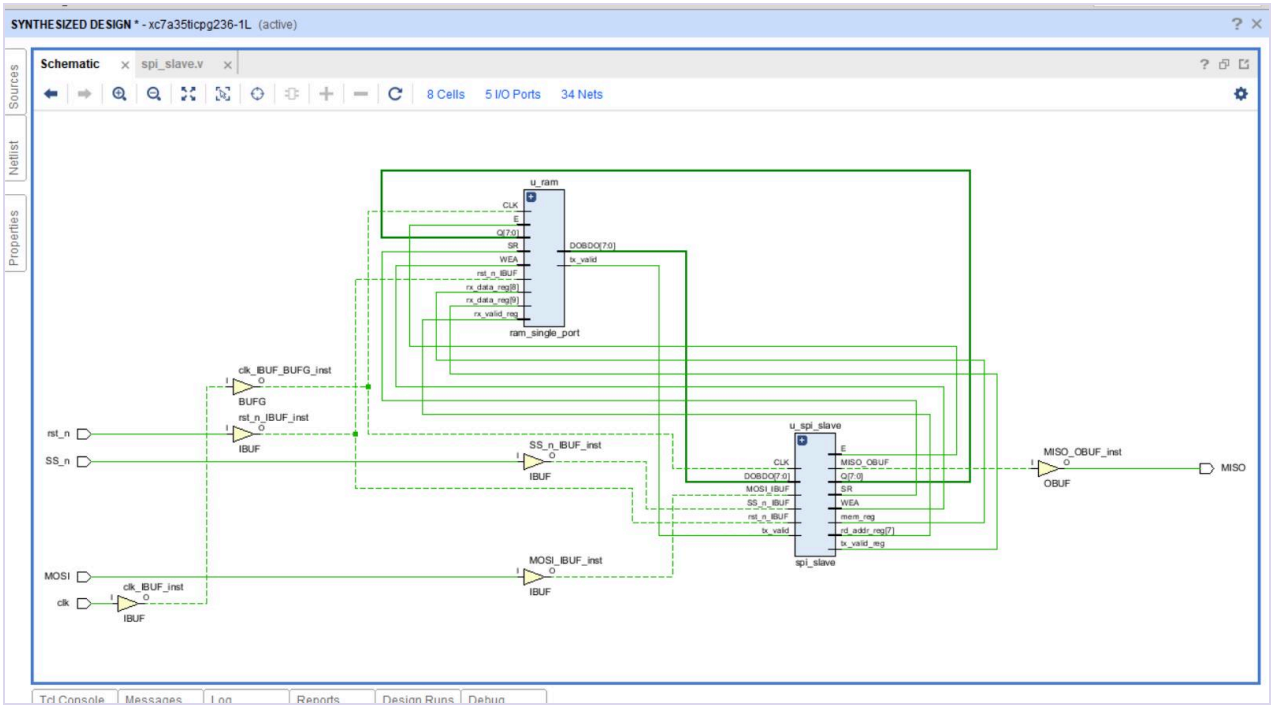


Figure 16. *Sequential: schematic after synthesis (8 cells, 5 I/O ports, 34 nets).*

6.3 Sequential: synthesis report, timing and critical path

```
99  ----------------------------------------------------------------------------------------------------
100 |                State |                New Encoding |                Previous Encoding
101 ----------------------------------------------------------------------------------------------------
102 |              S_IDLE |                         000 |                             000
103 |           S_CHK_CMD |                         001 |                             001
104 |             S_WRITE |                         010 |                             010
105 |         S_READ_DATA |                         011 |                             100
106 |          S_READ_ADD |                         100 |                             011
107 ----------------------------------------------------------------------------------------------------
108 INFO: [Synth 8-3354] encoded FSM with state register 'state_reg' using encoding 'sequential' in module 'spi_slave'
109 ---------------------------------------------------------------------
```

Figure 17. *Sequential: synthesis log showing the FSM encoding used (`'sequential'` for `state_reg` in `spi_slave`).*

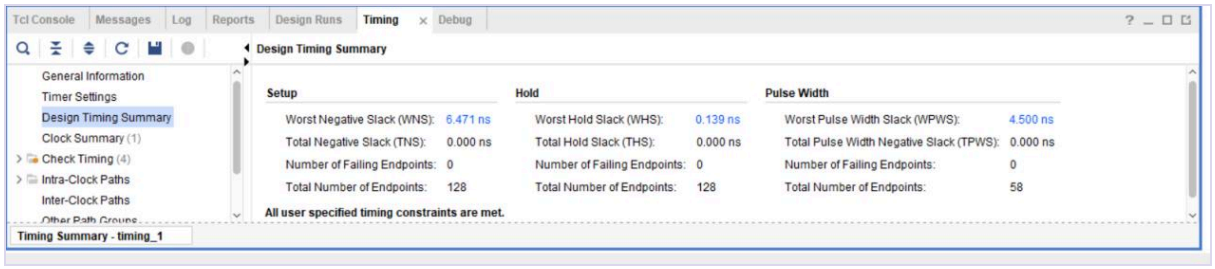


Figure 18. *Sequential: post-synthesis design timing summary.*

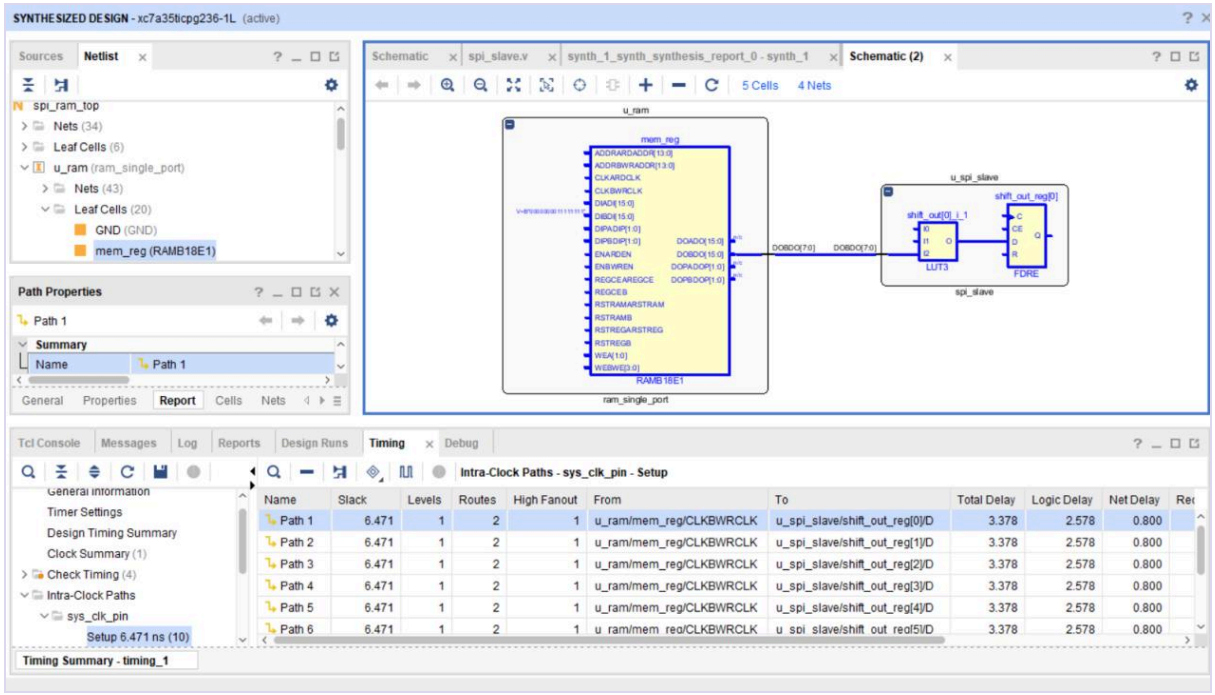


Figure 19. *Sequential: critical path highlighted in the schematic, with the intra-clock path report.*

07 Implementation

Vivado placed and routed design, per FSM encoding

Each encoding was implemented on the same device (**xc7a35ticpg236-1L**) with the same constraints. For each one this section shows the utilization report (hierarchy and summary), the design timing summary and the device view.

Constraint file

constraints_2.xdc · 31 lines

```
## Per the assignment: rst_n, SS_n, MOSI -> 3 switches, MISO -> 1 LED.
## ============================================================================

## ---- Clock: on-board 100 MHz oscillator ----
set_property -dict { PACKAGE_PIN W5  IOSTANDARD LVCMOS33 } [get_ports clk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports clk]

## ---- rst_n -> switch SW0 ----
## Board switch is active-high (up = 1). Since rst_n is active-LOW,
## flip SW0 DOWN to assert reset, UP for normal operation.
set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports rst_n]

## ---- SS_n -> switch SW1 ----
## Same idea: SW1 DOWN (0) selects the slave, SW1 UP (1) deselects it.
set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports SS_n]

## ---- MOSI -> switch SW2 ----
set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports MOSI]

## ---- MISO -> LED0 ----
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports MISO]

## ============================================================================
## General configuration options (kept as in the board's reference XDC)
## ============================================================================
set_property CONFIG_VOLTAGE 3.3 [current_design]
set_property CFGBVS VCCO [current_design]

set_property BITSTREAM.GENERAL.COMPRESS TRUE [current_design]
set_property BITSTREAM.CONFIG.CONFIGRATE 33 [current_design]
set_property CONFIG_MODE SPIx4 [current_design]
```

Post-implementation summary

Encoding	LUT	FF	BRAM	IO	WNS	WHS	WPWS
One-hot	1289 (6.20%)	1973 (4.74%)	1 (2.00%)	5	2.812 ns	0.012 ns	3.750 ns
Gray	1293 (6.22%)	1971 (4.74%)	1 (2.00%)	5	1.789 ns	0.029 ns	3.750 ns
Sequential	1292 (6.21%)	1971 (4.74%)	1 (2.00%)	5	1.725 ns	0.068 ns	3.750 ns

All user-specified timing constraints are met. Utilization totals include the debug cores (`dbg_hub`, `u_ila_0`) shown in each hierarchy report.

7.1 One-hot: utilization and timing

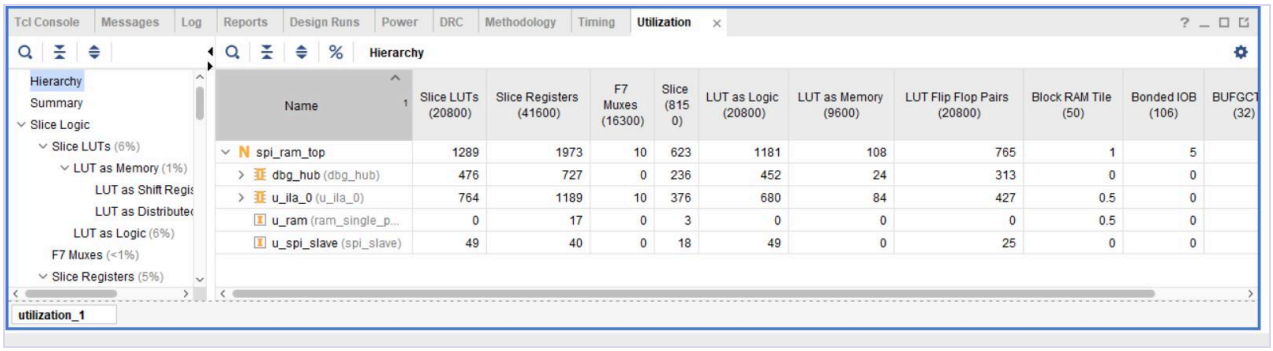


Figure 20. *One-hot: utilization report, hierarchy view.*

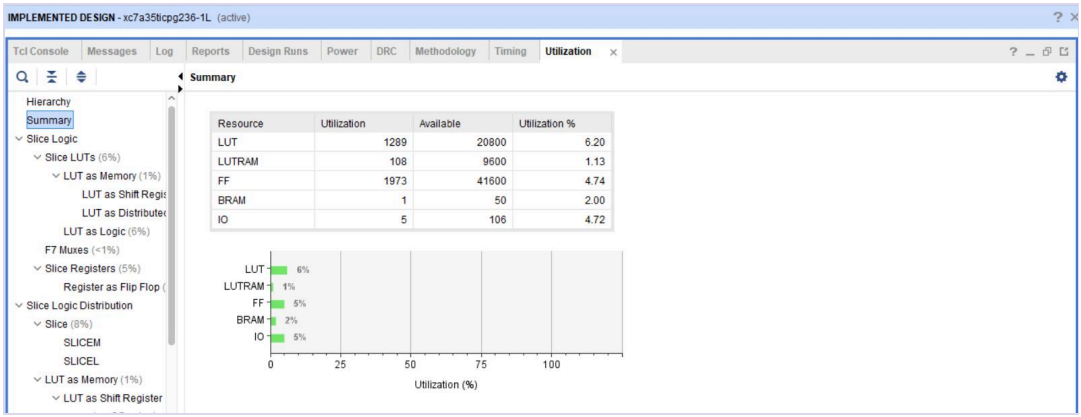


Figure 21. *One-hot: utilization report, summary view.*

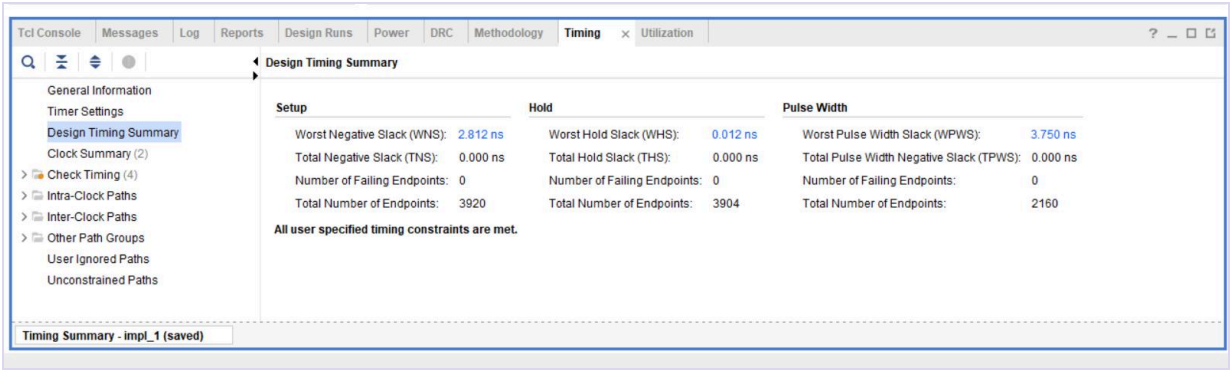


Figure 22. *One-hot: post-implementation design timing summary.*

7.1 One-hot: FPGA device view

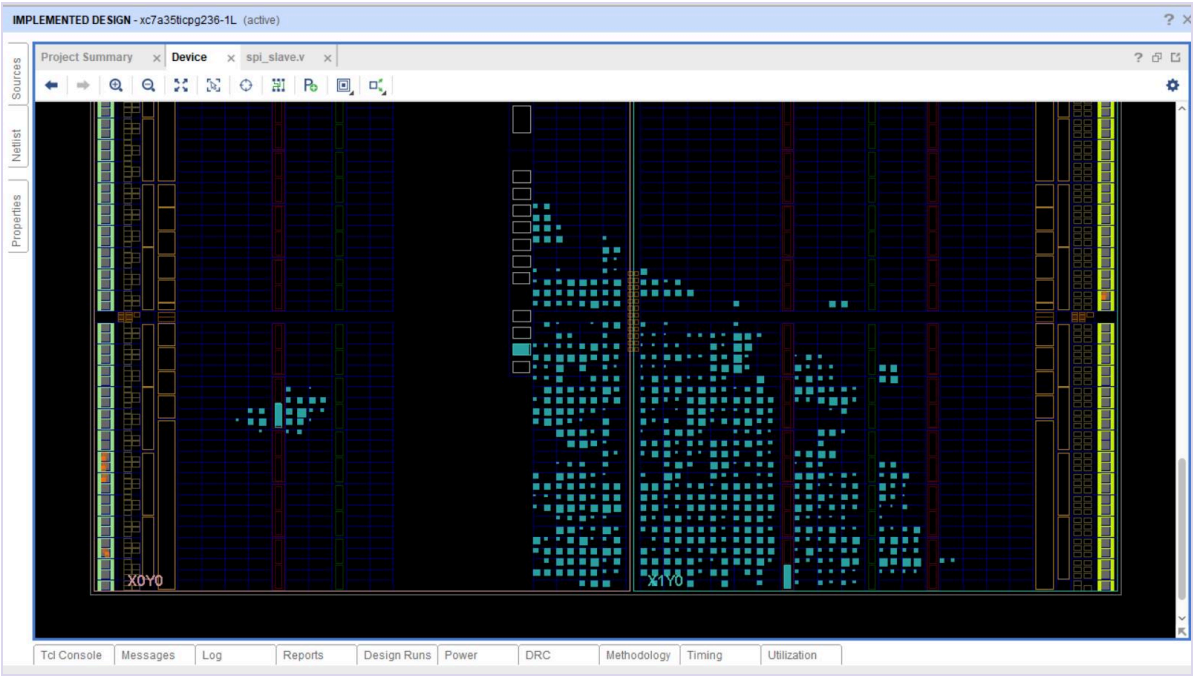


Figure 23. *One-hot: implemented design, device view (placed cells).*

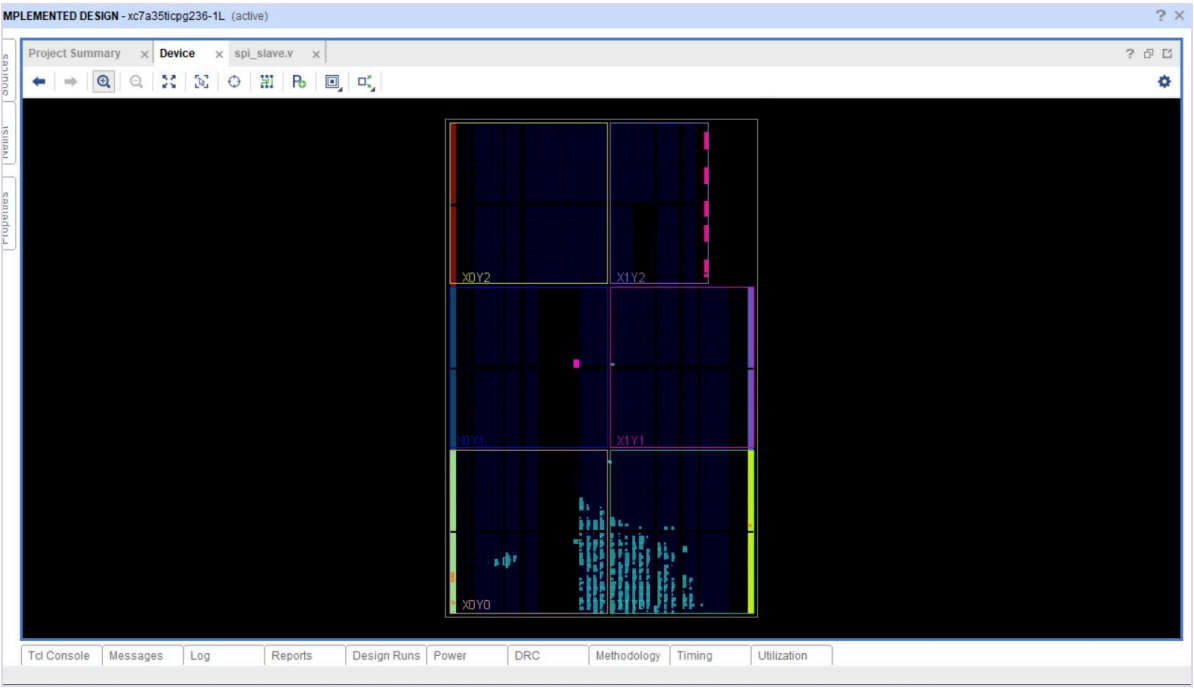


Figure 24. *One-hot: implemented design, full device view.*

7.2 Gray: utilization and timing

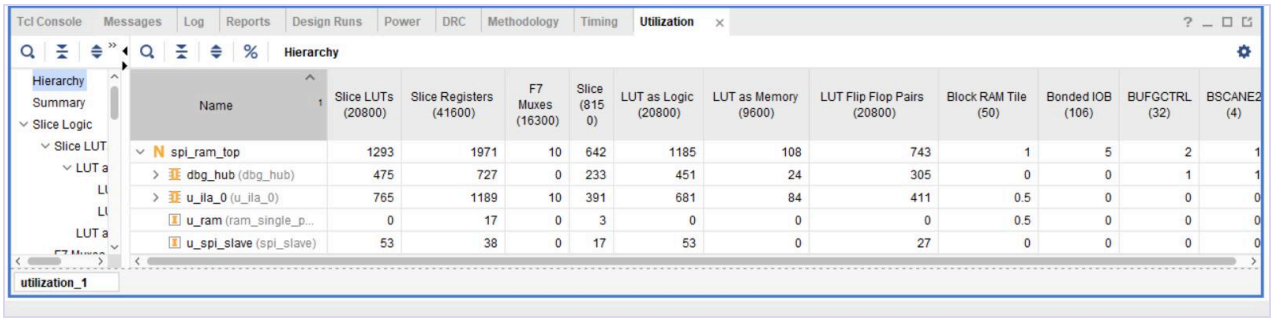


Figure 25. *Gray: utilization report, hierarchy view.*

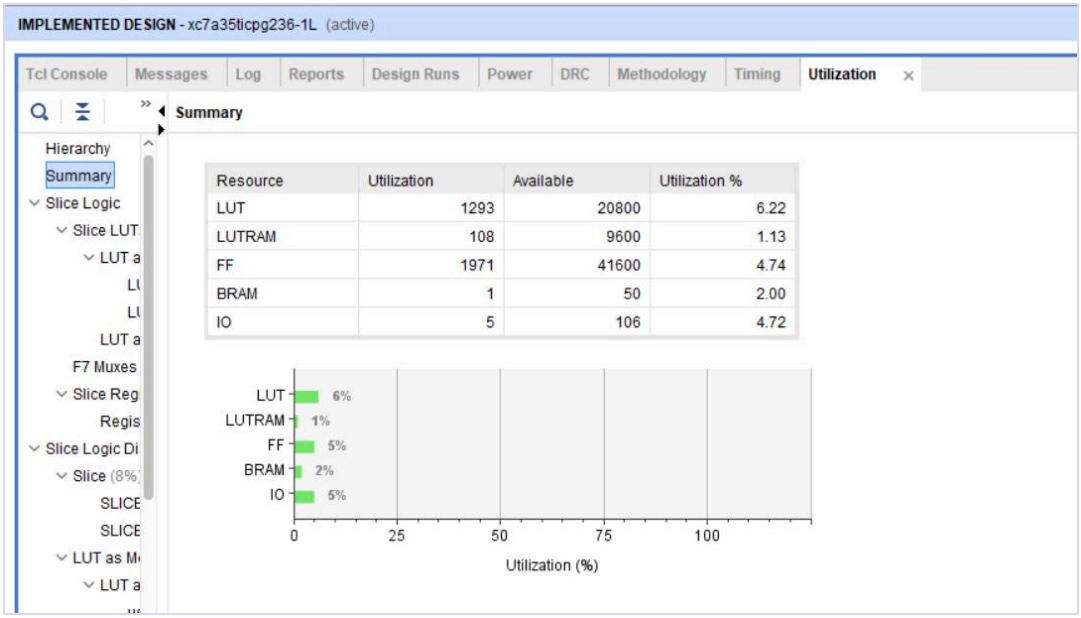


Figure 26. *Gray: utilization report, summary view.*

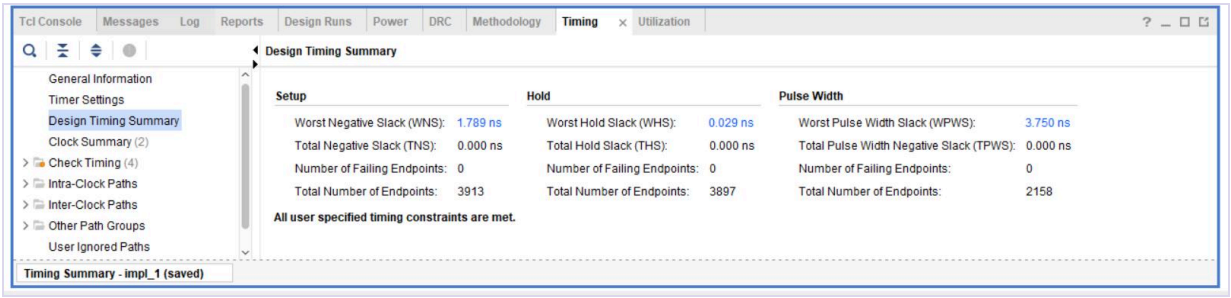


Figure 27. *Gray: post-implementation design timing summary.*

7.2 Gray: FPGA device view

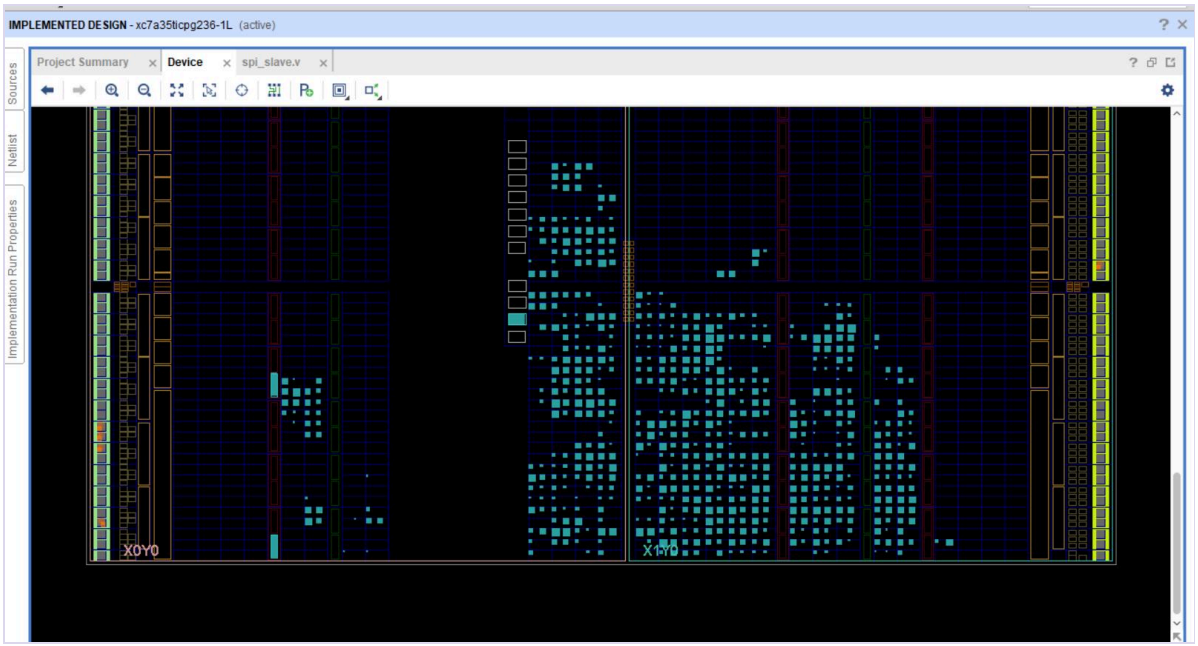


Figure 28. *Gray: implemented design, device view (placed cells).*

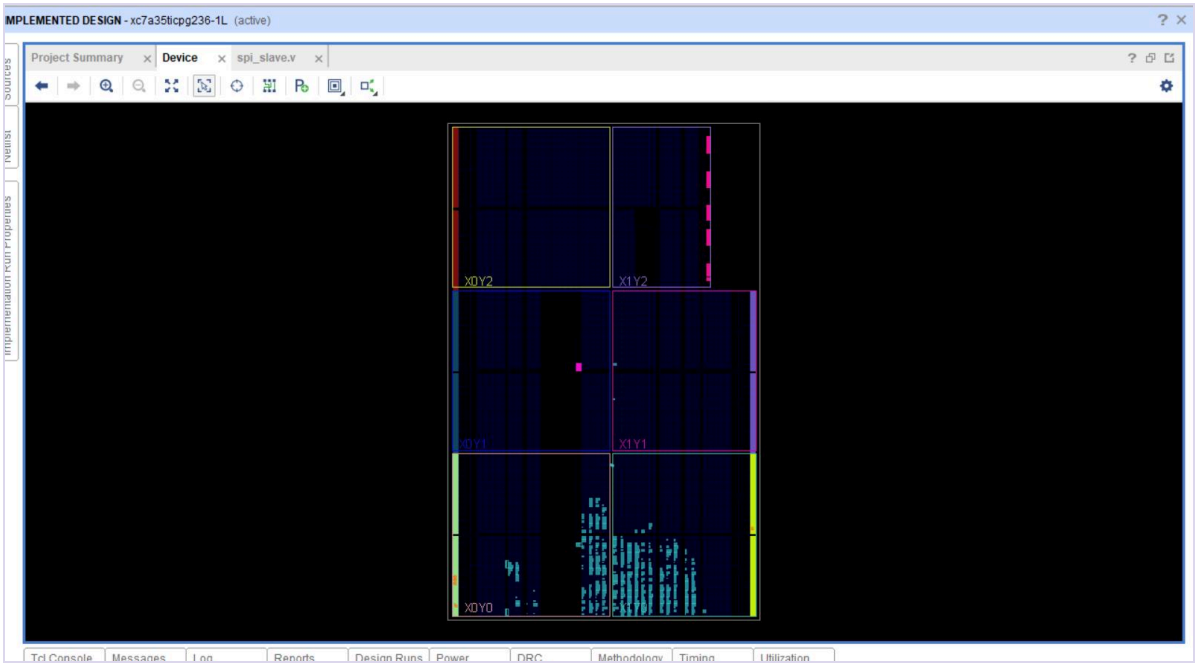
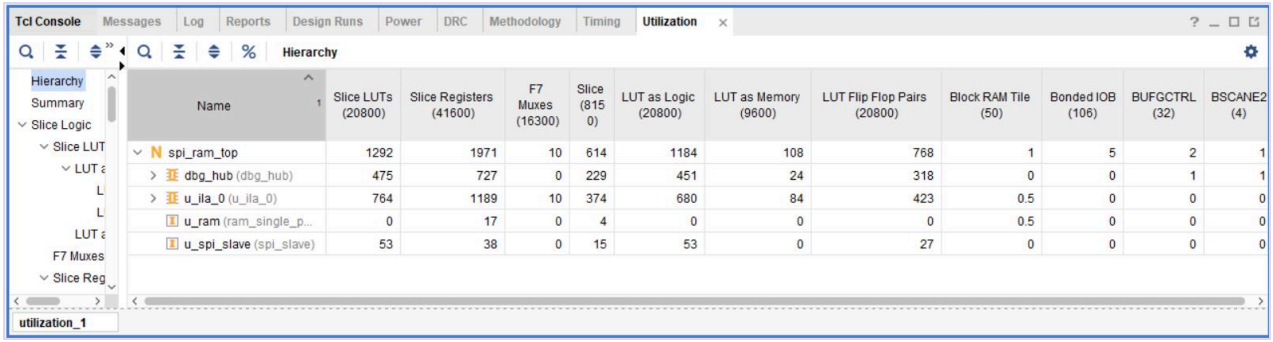


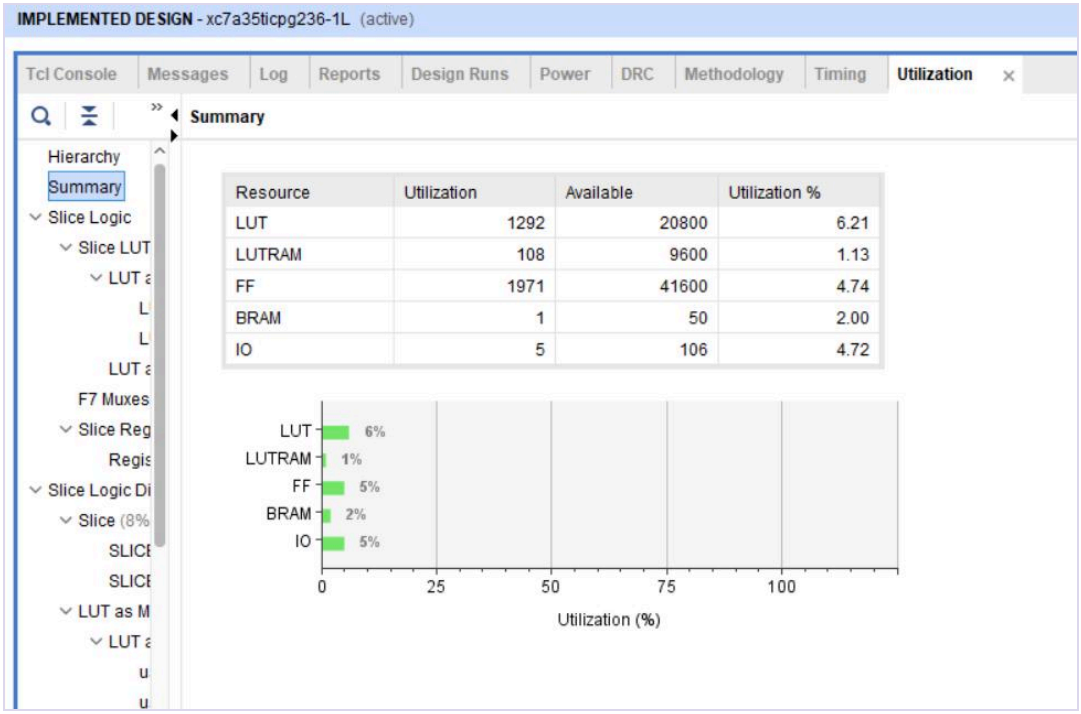
Figure 29. *Gray: implemented design, full device view.*

7.3 Sequential: utilization and timing



Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	Slice (8150)	LUT as Logic (20800)	LUT as Memory (9600)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)	BSCANE2 (4)
spi_ram_top	1292	1971	10	614	1184	108	768	1	5	2	1
dbg_hub (dbg_hub)	475	727	0	229	451	24	318	0	0	1	1
u_ila_0 (u_ila_0)	764	1189	10	374	680	84	423	0.5	0	0	0
u_ram (ram_single_p...	0	17	0	4	0	0	0	0.5	0	0	0
u_spi_slave (spi_slave)	53	38	0	15	53	0	27	0	0	0	0

Figure 30. *Sequential: utilization report, hierarchy view.*



Resource	Utilization	Available	Utilization %
LUT	1292	20800	6.21
LUTRAM	108	9600	1.13
FF	1971	41600	4.74
BRAM	1	50	2.00
IO	5	106	4.72

Figure 31. *Sequential: utilization report, summary view.*

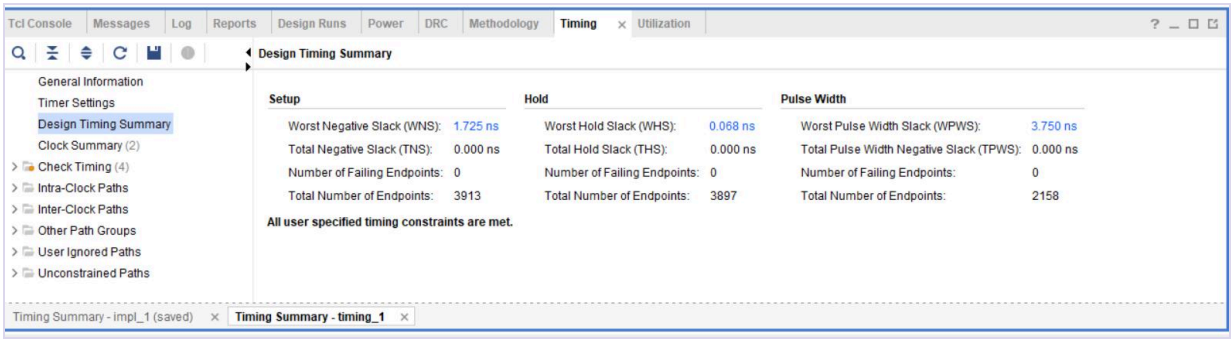


Figure 32. *Sequential: post-implementation design timing summary.*

7.3 Sequential: FPGA device view

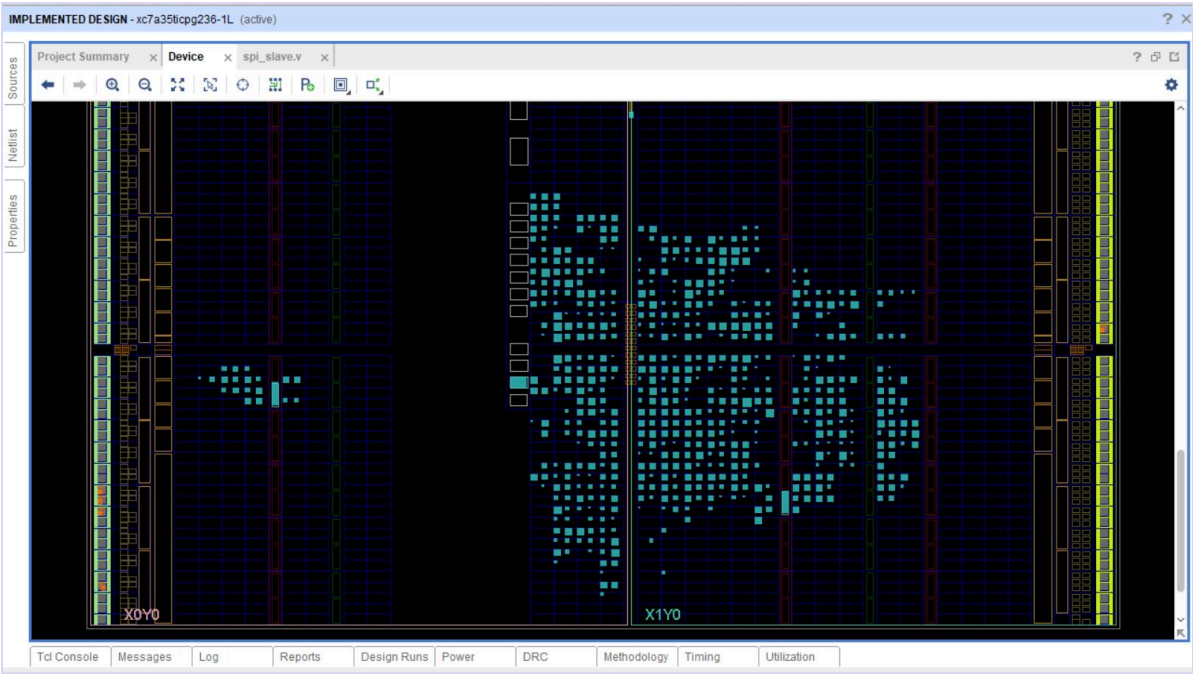


Figure 33. *Sequential: implemented design, device view (placed cells).*

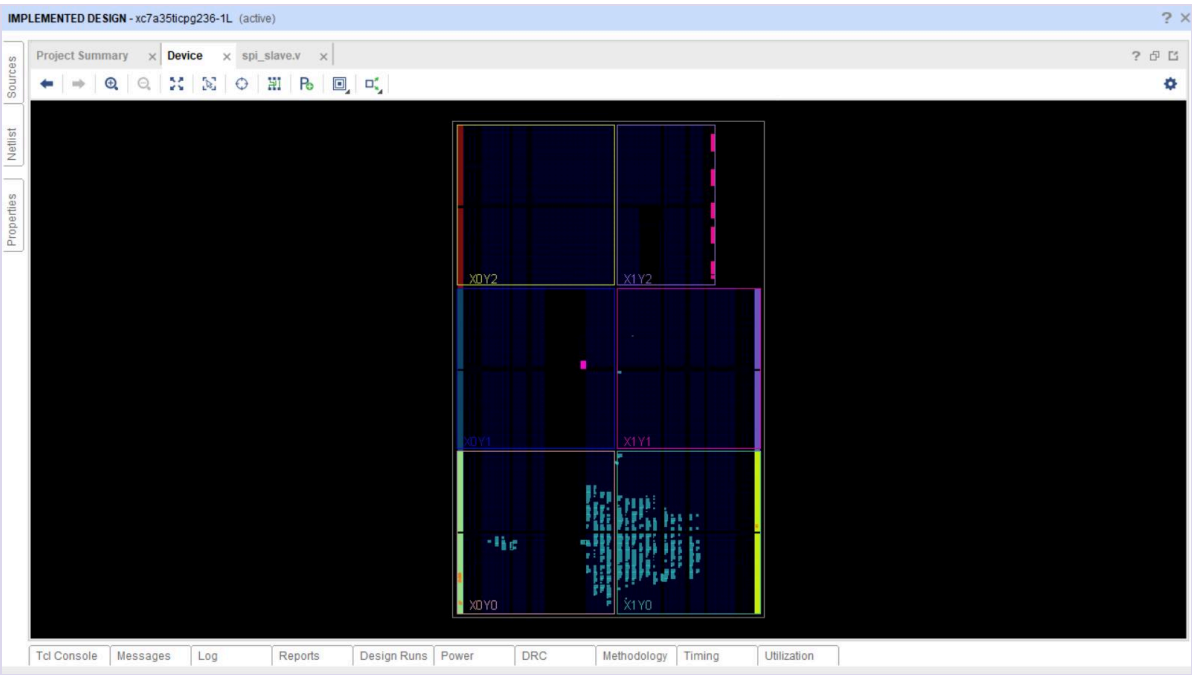


Figure 34. *Sequential: implemented design, full device view.*

08 FSM Encoding Selection

Comparing one-hot, gray and sequential after implementation

Encoding	WNS (setup)	WHS (hold)	spi_slave LUT / FF	Top LUT / FF
One-hot	**2.812 ns**	0.012 ns	49 / 40	1289 / 1973
Gray	1.789 ns	0.029 ns	53 / 38	1293 / 1971
Sequential	1.725 ns	0.068 ns	53 / 38	1292 / 1971

Worst negative slack (setup), post-implementation



Higher is better. Clock period 10.00 ns.

Selected encoding: One-hot

- One-hot gives the **largest setup slack** after implementation: 2.812 ns against 1.789 ns (gray) and 1.725 ns (sequential).
- It needs the **fewest LUTs** in the FSM instance (49 against 53) and the lowest total LUT count (1289), for two extra state flip-flops (40 against 38 in `spi_slave`).
- Its worst hold slack (0.012 ns) is the smallest of the three but still positive; there are no failing endpoints and all timing constraints are met for every encoding.
- The RTL carries the matching attribute: `(* fsm_encoding = "one_hot" *)` on the state register.